Qwen

# A Schema-Centric Architecture: Formalizing Objectives, Unifying Hazards, and Standardizing Rules for Conker: Live & Uncut

## Formalization of Generic Delivery-Based Objectives

The core directive to formalize the "feed victim to objective" and "pickup and cook" mechanics from Raptor Temple into generic objective types within the shared entities schema represents a significant architectural evolution for the Conker: Live & Uncut multiplayer project [1]. This initiative moves beyond map-specific implementation details to establish a foundational, reusable pattern that will govern a wide variety of scoring mechanics across the game's lifespan. The underlying principle is to identify and codify recurring gameplay loops into declarative schema entries, thereby transforming imperative, procedural code into a more flexible, data-driven system. This approach ensures that future maps can adopt complex behaviors without requiring bespoke coding, promotes consistency in player experience, and provides a clear, stable contract for both human designers and AI-assisted development tools [7] [33]. The decision to implement this now, even with its first usage confined to Raptor Temple, is a forward-looking investment in the project's long-term maintainability and scalability [1].

The analysis of existing game modes reveals that the fundamental pattern of "deliverable/consume-at-objective" is already established but not yet fully abstracted into a generic type. For instance, gas canisters in Fortress serve as a prime example of this mechanic; a player must pick one up, carry it through contested territory, and deliver it to a designated arming point at an enemy base to trigger a significant event [9]. Similarly, heavy-carry items such as Bazookas or Money Bags impose a movement penalty and are scored when brought to a specific objective location [17]. The "Alien Egg" objective in Alien Base further refines this pattern, where damaging or destroying the egg triggers alien spawns or escalates hazards [3]. These mechanics share a common structure: a player entity interacts with a pickup object, carries it through a high-risk environment, and performs a turn-in action at a designated objective volume to achieve a score increment or other game-altering effect. Raptor Temple's two primary scoring loops

—the act of grabbing a caveman and feeding him to the Baby Dino, and the process of stealing an egg and cooking it at a pan—are structurally identical to this established pattern [11] [20]. Both involve picking up a deliverable item and interacting with a specific world actor to score. Therefore, formalizing them is not an invention of a new concept, but rather the recognition and abstraction of a pattern that has been implicitly present in the game's design philosophy since its inception.

To formalize these mechanics, the shared entities schema (`entities_v1.schema.json`) must be extended to include new, generic objective types. These types would serve as templates for actors placed in a map's entities file (`raptor_temple_entities_v1.json`, `blood_count_entities_v1.json`, etc.). The proposed additions would be `objective_type: baby_dino_feeder` and `objective_type: egg_cook_pan`. Each of these types would be a thin specialization of a generic objective base class, inheriting core functionalities like replication, health management, and score event broadcasting [29]. The key distinction lies in their specific behaviors, which would be defined in their respective engine implementations (Unreal C++, Unity C#, Godot GDScript). For the `baby_dino_feeder` type, the behavior would be to react to a delivered "victim" pawn. When a pawn tagged as a "victim" (e.g., a caveman in a specific ASID) enters the objective's volume, the system would validate the interaction, award a score to the delivering team, and initiate a "hard-lock execution state" on the victim pawn, effectively killing them [1]. This logic mirrors how the Alien Egg currently operates, reacting to damage or destruction events [3]. Conversely, the `egg_cook_pan` type would function as a turn-in volume for a pickup item. A player carrying a raw egg (an object with a specific ASID, e.g., `ASID_EGG_PICKUP`) would interact with the pan, triggering an animation and a score event before despawning the egg [1]. This directly parallels the Heist bag mechanic and the gas canister delivery [9]. By defining these behaviors in a central, shared component, any map can simply place an objective actor with one of these types in its entities file, relying on the pre-built logic for all the complex interactions. This eliminates the need for developers to copy-paste code between maps and reduces the potential for inconsistencies and bugs [5].

The strategic advantages of this schema-first approach are profound. First and foremost, it future-proofs the codebase. A designer working on a future map, such as a hypothetical "Blood Count Sacrifice" variant, could easily create a new objective type like `objective_type: blood_sacrifice_altar`. Because the underlying architecture is built on the same principles as the Baby Dino feeder, the developer would only need to write a small amount of map-specific code to define the altar's unique visual effects and final kill condition, while reusing the vast majority of the logic for victim detection, scoring, and state management. This drastically lowers the barrier to entry for creating

new content and accelerates development cycles. Second, it enhances transparency and predictability, which is critical for both debugging and AI-assisted development. An AI assistant tasked with generating code for a new map would have a stable, documented set of objective types to work with. It would know, for example, that placing an entity with `objective_type: egg_cook_pan` requires a corresponding `egg_pan` roletag in the grid file and a valid cooking interaction context button prompt [1]. This creates a clear contract between the schema and the implementation, preventing the AI from inventing its own inconsistent patterns and making the generated code more reliable and easier to integrate [19] [30]. Finally, this approach enforces consistency across all modes and maps. All instances of a "feed victim" objective would share the exact same logic for scoring animations, sound cues, and network replication, ensuring that players have a consistent mental model regardless of which map they are playing. This coherence is essential for building a polished and professional-feeling game. The preservation of the original N64-era gameplay feel is also supported, as these mechanics are direct descendants of classic asymmetric modes like Raptor, and formalizing them preserves their intended identity while making them more robust [12] [13].

Furthermore, the formalization extends beyond just the objective actors themselves to encompass the abilities required to perform the actions. The user's clarification specifies that these mechanics should be implemented while adhering to the "no classes" philosophy of Conker: Live & Uncut [1]. Abilities like the "Raptor Pounce" and the Caveman's toolkit are not tied to permanent character classes but are instead temporary states driven by Animation State ID (ASID) entries and powered by pickups . For Raptor Pounce, this would be an ASID like `ASID_RAPTOR_POUNCE_EXEC`, which grants a short-range lunge attack and locks input during the execution animation . A player could gain this ability by picking up a special item or, more likely, by being on the "Team Raptor" faction, which is spawned with the ability enabled by default. Similarly, the Caveman's abilities are modeled as ASIDs: `ASID_CAVEMAN_EGG_CARRY` for holding an egg, which imposes a heavy-carry penalty, and `ASID_CAVEMAN_COOKING` for performing the cooking animation . Technically, any pawn could enter these states if the mode rules allowed it, but the default configuration ensures the intended asymmetry is maintained without violating the no-classes contract . By defining these ASIDs in a central registry and providing cross-engine helper components, the project ensures that these temporary abilities are implemented consistently and can be referenced reliably by any part of the system, including AI-directed logic . This tight coupling of objective types and ability states within a centralized schema creates a cohesive and extensible framework for designing asymmetric and objective-based gameplay.

| Feature | Current Implementation (Raptor Temple) | Proposed Formalized Implementation |
|---|---|---|
| **Objective Definition** | Map-specific entity definitions and logic within C++/GDScript files. | Generic `objective_type` entries (`baby_dino_feeder`, `egg_cook_pan`) in the shared entities schema. |
| **Scoring Logic** | Hardcoded behavior within the objective's implementation file. | Reusable logic within a generic objective base class, specialized per `objective_type`. |
| **State Management** | Implicit state tied to the map's scripting. | Explicit state transitions managed via the shared ASID registry. |
| **Player Interaction** | Specific context prompts and interaction scripts. | Standardized interaction prompts handled by the generic objective component. |
| **Future Extensibility** | Requires custom code for each new mechanic or map. | New mechanics can be added by defining new `objective_type`s and associated ASIDs in the schema. |
| **AI-Assisted Development** | High ambiguity due to lack of standardized names and contracts. | Clear, stable names and contracts provided by the schema, improving reliability. |

This structured approach ensures that the transition from a collection of individual maps to a cohesive game engine is seamless. By capturing the essence of Raptor Temple's core loop and elevating it to a generic, schema-defined pattern, the project lays a solid foundation for all future multiplayer content. It embodies a mature software engineering philosophy: build once, use many times. The initial effort to formalize these objectives pays dividends in reduced technical debt, accelerated development, and a more robust, predictable, and enjoyable gaming experience for players across all maps and modes.

## Unification of Hazard Systems Through Data-Driven Profiles

The second major initiative, the unification of hazard handling across Fortress and Alien Base, addresses a critical point of fragmentation in the codebase. Currently, each map implements its own distinct hazard logic, leading to duplicated code, divergent tuning, and increased maintenance overhead. Fortress employs a `GasWave` actor controlled by the `AGasCanister` state machine, which activates upon the delivery of a gas canister to an enemy base [9]. Alien Base, in contrast, utilizes a `AlienBaseAirlockController` state machine to toggle the enabled state of separate `AAlienBaseVolumeHubFloorGas` and `AAlienBaseVolumeSublevelAcid` hazard volumes [3]. While functionally similar—both deal damage over time (DoT) to players who enter their volume—they are implemented as entirely separate classes with potentially overlapping but inconsistently applied logic. The proposed solution is to extract this common functionality into a single, shared `HazardVolume` base class, driven by data-driven JSON profiles rather than hardcoded values or compile-time assignments.

This approach is fundamentally aligned with the project's overarching goal of creating an engine-agnostic, data-centric framework that prioritizes flexibility and scalability [4] [26].

The core of this unification strategy is the creation of the `HazardVolume` base class. This class would serve as a template for all environmental hazards in the game, encapsulating the universal logic that applies to any DoT effect. Its primary responsibilities would include: applying damage at regular intervals, checking for immunity based on the target pawn's active ASIDs, toggling its physical and visual representation (e.g., particle effects), and communicating its state to controlling systems. The most critical aspect of this class is its reliance on a profile identifier to determine its specific properties. Instead of having a `HazardVolume` subclass for every possible hazard (e.g., `GasHazardVolume`, `AcidHazardVolume`, `ElectricHazardVolume`), there would be one `HazardVolume` class that reads its configuration from a central JSON file, such as `config/hazards/hazard_profiles_v1.json` [1]. This profile would map a unique `hazard_type` string (e.g., "hub_gas_v1", "sublevel_acid_v1") to a set of parameters. This shift from imperative, class-based differentiation to declarative, data-driven configuration is the key to achieving true unification. The same piece of code, the `HazardVolume` component, can become a different hazard simply by loading a different profile, dramatically reducing code duplication and simplifying the entire system [22].

The parameters managed within these data-driven profiles would cover all aspects of a hazard's behavior, allowing for complete control over its properties without touching any code. Key parameters would include:

- `damage_per_second`: A float value defining the rate at which the hazard deals damage. For example, the hub gas might be configured at 60 HP/s, while the sublevel acid is tuned to 90 HP/s, reflecting its greater lethality [1].
- `tick_interval`: A float value specifying the frequency (in seconds) at which the damage calculation is evaluated. A shorter interval results in smoother damage application but may increase CPU overhead.
- `immunity_asids`: A list of ASID integer values that grant immunity to the hazard. This is crucial for respecting execution states, as a pawn locked in an animation like `ASID_RAPTOR_POUNCE_EXEC` should not take additional environmental damage [1] [35]. The shared rule is that if the target pawn's active ASID set intersects with this list, the damage application is skipped for that tick [1].
- `visual_effect_id` and `audio_cue_id`: References to the specific particle effects, materials, and sounds that should be played when the hazard is active, allowing for distinct sensory feedback for each hazard type.

- `activation_controller`: A reference to the state machine or controller responsible for enabling and disabling the volume (e.g., the `AlienBaseAirlockController` or the `AGasCanister` logic).

This data-driven model offers immense flexibility compared to a static, compile-time assignment system. With the proposed runtime-switching capability, a single `HazardVolume` component in a map's entities file can be dynamically configured to behave differently depending on the active mode profile or even the difficulty setting [1]. For instance, a level designer could configure the `AlienBaseAirlockController` to activate "hub_gas_v1" during a standard match but "hub_gas_hell_mode_v1" during a special event, where the latter profile has a higher `damage_per_second` and a shorter `tick_interval`. This allows for easy balancing and the creation of varied gameplay experiences without requiring developers to write new code or modify existing controllers [1]. Furthermore, this system naturally supports the creation of new hazard types in the future. To add a new hazard, such as electrified floor tiles, a developer would simply need to create a new entry in the `hazard_profiles_v1.json` file with the desired parameters and then instantiate a `HazardVolume` actor in the map, assigning it the new hazard type's profile ID. The `HazardVolume` base class already contains all the necessary logic to handle the new hazard's activation and damage application. This contrasts sharply with the current fragmented approach, where adding a new hazard would require writing a new C++/C# class, integrating it with the relevant state machines, and updating any related UI or AI logic.

The user's explicit preference for a runtime-configurable system over a static one underscores the importance of this flexibility [1]. A statically assigned system, where a hazard's properties are determined at compile time or during map build, would still offer some benefits but would be far less adaptable. It would tie the hazard's behavior tightly to a specific map or asset, making it difficult to reuse components across different contexts. The runtime model, driven by JSON profiles, treats hazard properties as variables that can be changed on the fly. This aligns perfectly with modern game development practices that prioritize modularity and data-over-code philosophies. It also simplifies the CI/CD pipeline and editor tooling. Instead of needing complex scripts to parse and bake in different hazard settings, the game engine simply loads the appropriate JSON file at runtime and configures the hazard components accordingly. This makes the development process more transparent and easier to manage, especially when multiple developers or AI agents are working on different aspects of the game [29]. The `grid2scene` tool, which already converts grid and entities JSON files into scene objects, can be taught to resolve these profile IDs, ensuring that the connections between controllers, objective volumes, and hazard profiles are validated and consistent across all

maps [25]. Ultimately, this unification effort transforms the hazard system from a collection of disparate, brittle pieces into a robust, scalable, and highly configurable subsystem that serves as a cornerstone for the entire game's environmental challenge design.

## Standardization of Zombie Kill Mechanics with Controlled Exceptions

The third pillar of this architectural enhancement focuses on the specific behavioral rules for The Blood Count's zombie crawl state, aiming to strike a delicate balance between preserving a core gameplay element from the source material and providing necessary design flexibility. The central mechanic to be addressed is the requirement that zombies, particularly those in a crawling state, can only be killed by precise headshots or close-range shotgun blasts . This rule is a direct translation of a defining characteristic from the original "Spooky" chapter of *Conkers Bad Fur Day*, where Gregg explicitly instructs the player that body shots are ineffective against zombies . Implementing this as a universal baseline rule is paramount for maintaining the intended tense, deliberate gameplay feel of The Blood Count. However, to avoid hindering overall gameplay flow and variety, the system must also allow for a small number of explicitly defined exceptions. This approach ensures that the core mechanic remains intact and recognizable while giving designers a controlled way to break the rule when creatively justified, all managed through a centralized and transparent system.

The baseline rule is non-negotiable: a zombie that has entered its crawl state due to taking limb or torso damage must not die from subsequent body hits . The moment a zombie accumulates enough body damage to fall over, its hitbox and behavior profile should change. It would transition to a crawl animation (a distinct ASID, e.g., `ASID_ZOMBIE_CRAWL`), drop its movement speed, and become significantly harder to dispatch . Any damage dealt to its body or limbs in this state would have no effect on its health, causing it to remain as a persistent threat on the battlefield—a crawling carpet of enemies that forces players to either retreat or take careful aim with powerful weapons like the shotgun or strong firearms [13]. This creates a specific tactical dynamic where spraying fire into a group of crawling zombies is a waste of ammunition and time, whereas carefully targeting heads or firing point-blank with a shotgun yields a swift result. From an implementation standpoint, this logic should be integrated directly into the shared Damageable/ASID system. A simple rule within the damage application module could state: "if the incoming hit is to the body/torso, did not originate from a

shotgun at point-blank range, and the target's health is below the 'crawling threshold', force the target into the `ASID_ZOMBIE_CRAWL` state instead of applying damage." This centralized rule ensures that all three engines (Unreal, Unity, Godot) reuse the exact same logic table, guaranteeing consistent behavior across the board .

While the headshot-only rule forms the bedrock of the mechanic, rigidly enforcing it under all conceivable circumstances would lead to frustrating gameplay scenarios. For example, a player who has just been caught in a hard-lock execution animation (like being fed to the Baby Dino) should not have to worry about simultaneously aiming for a zombie's head to survive. To address this, the system must incorporate explicitly defined exceptions. The user's directive clarifies that these exceptions should be deliberately whitelisted and encoded centrally, rather than being scattered as ad-hoc checks in various parts of the codebase [1] . The most logical and impactful exception is tied to ASID states that represent a "hard-locked" state for the player pawn. If a pawn is in an ASID such as `ASID_RAPTOR_POUNCE_EXEC` or any other cinematic execution animation, the kill conditions for nearby threats should automatically adjust. In this state, the pawn is immobile and cannot take evasive action, making them a perfect candidate for an automatic kill from any source of damage, including a crawling zombie. This maintains the narrative weight of the cinematic moment without punishing the player for failing to aim perfectly during a scripted sequence.

Another potential exception could be granted to specific, high-tier weapons. A rare, powerful weapon pickup (e.g., a rocket launcher or a flamethrower) could be given a special property in the weapon definition that allows it to bypass the crawling state and kill the zombie instantly with a body shot . This would reward players who successfully acquire and utilize these powerful tools, giving them a decisive advantage in clearing groups of crawling zombies. By encoding these exceptions centrally—in the ASID registry or a dedicated damage rules schema—the system remains transparent and maintainable. A designer looking to understand why a particular interaction is valid can consult a single source of truth, rather than hunting through dozens of script files. This is vastly superior to the alternative of scattering `if (weapon == ROCKET_LAUNCHER)` or `if (pawn.ASID == EXECUTION_STATE)` checks throughout the damage application logic, which would create a tangled web of dependencies and make the system brittle and difficult to debug [4] . Centralizing the rules also facilitates AI-assisted development; an AI assistant generating code for a new weapon or ability would know exactly where to look to add an exception to the crawling zombie rule, ensuring consistency and adherence to the established design contract [19] .

The implementation of this system would rely heavily on the existing ASID infrastructure, which is designed to manage temporary character states and abilities . The crawl state

itself is a simple ASID toggle. The exceptions, however, require a more sophisticated rule-checking mechanism. When damage is applied, the system first determines the target's state. If the target is a crawling zombie, the damage application logic queries a central rules engine. This engine evaluates the incoming damage's source (the attacking pawn and the weapon used) against the predefined exception list. If a match is found—for instance, the attacker is in a death-animation ASID, or the attacking weapon has the "ignore_crawl_state" flag—the system proceeds to apply the damage and register a kill. Otherwise, the damage is absorbed by the zombie's remaining health pool, and it continues to crawl. This layered approach, combining a universal baseline rule with a centrally managed exception policy, provides the ideal blend of fidelity to the source material and creative freedom for game designers. It ensures that the tense, headshot-focused gameplay of Spooky is preserved as the default experience, while still allowing for moments of heightened intensity or powerful player capabilities that temporarily suspend that rule. This methodical and transparent approach to rule definition is a hallmark of well-engineered systems and is essential for the long-term success and expandability of the Conker: Live & Uncut multiplayer codebase.

## Strategic Synthesis of System-Wide Improvements

The three interconnected initiatives detailed in this report—formalizing generic objectives, unifying hazard systems, and standardizing zombie kill mechanics—are not isolated tasks but components of a coherent, overarching strategy to evolve the Conker: Live & Uncut codebase from a series of map-specific implementations into a scalable, data-driven game engine. Collectively, these changes embody a fundamental shift in architectural philosophy, moving decisively away from procedural, imperative code towards a declarative, schema-driven paradigm. This strategic direction is aimed at maximizing scalability, enhancing long-term maintainability, and fostering a development environment that is both transparent and conducive to future innovation, including AI-assisted development. The common thread weaving through all three pillars is the principle of centralization: consolidating logic, rules, and configuration into authoritative sources (JSON files and a shared schema) that can be consumed by a variety of systems, thereby reducing redundancy and increasing predictability.

First, the formalization of delivery-based objectives like "feed victim" and "pickup and cook" establishes a reusable pattern for a core category of gameplay mechanics. By defining these as generic `objective_type`s within the shared entities schema, the project creates a stable, documented contract that transcends any single map [1]. This

directly contributes to scalability by dramatically lowering the cost of implementing similar mechanics in future maps like Blood Count or Alien Base. Developers are no longer required to invent new systems or copy-paste code; they can simply extend the existing framework by referencing a known schema entry. This approach also directly supports maintainability. If a bug is discovered in the core logic of how a cooked egg scores points, the fix needs to be applied in only one place—the generic objective component—and the change propagates to every map that uses that objective type. This prevents the accumulation of technical debt that often plagues projects with fragmented, map-specific codebases [5]. Furthermore, this schema-first approach provides unparalleled transparency. The behavior of a game element is no longer hidden within a compiled binary or a cryptic script file; it is explicitly declared in a human-readable JSON file that serves as a living specification for the game's design [7] [18].

Second, the unification of the hazard system through a runtime-configurable `HazardVolume` base class and data-driven profiles tackles the issue of code duplication and inflexibility. The previous state, where Fortress and Alien Base had entirely separate hazard implementations, was a clear anti-pattern that created unnecessary complexity and made balancing and tuning a tedious, error-prone process [3] [9]. The introduction of a single base class whose properties are dictated by a JSON profile is a textbook example of leveraging polymorphism and data-driven design to solve this problem. The strategic advantage here is flexibility. By supporting runtime switching between different hazard profiles, the system can easily accommodate mode variants, difficulty levels, and rapid tuning cycles without requiring any code changes [1]. A level designer can dial up the severity of a hazard by simply editing a number in a JSON file, a task far simpler and safer than modifying C++ or C# code. This modular approach also simplifies the development pipeline and editor tooling, as demonstrated by the integration with the `grid2scene` utility, which can validate and manage the complex relationships between controllers, objectives, and hazard profiles [25]. The result is a more robust, resilient, and adaptable system for managing environmental challenges.

Third, the standardization of zombie kill mechanics with controlled exceptions demonstrates a mature understanding of balancing design fidelity with creative freedom. The decision to preserve the headshot-only rule as a universal baseline for crawling zombies is a commitment to honoring the specific, tense gameplay feel of the source material . However, the equally important decision to manage exceptions through a centralized, explicitly defined system shows a pragmatic awareness of real-world game design needs. Rather than allowing arbitrary, map-specific hacks to undermine the core mechanic, the proposal creates a controlled "escape hatch" that is transparent, maintainable, and predictable. This approach ensures that the system's behavior is always

grounded in a clear set of rules, whether it's the default case or a documented exception. It reinforces the project's commitment to building a well-documented, self-validating system where the intent behind the code is as important as the code itself [4].

In synthesis, these three initiatives form a virtuous cycle that strengthens the entire codebase. The formalized objectives provide a rich vocabulary of reusable components. The unified hazard system provides a flexible and scalable framework for environmental danger. The standardized kill rules provide a consistent and fair combat model. Together, they create a foundation upon which future developers, both human and artificial, can build confidently. They reduce the cognitive load required to understand and extend the game's systems, minimize the risk of introducing regressions, and accelerate the pace of content creation. This strategic pivot towards a data-centric architecture is not merely an optimization; it is a foundational upgrade that positions Conker: Live & Uncut for long-term success, ensuring that the spirit of the original games can be expressed through a modern, robust, and highly maintainable technological framework.

---

## Reference


1. https - cs.Princeton https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt
2. Steam 创意工坊::Ambrog's list of facking stuff https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=2352195111
3. Dictionary of Architecture and Building Construction - Academia.edu https://www.academia.edu/37128673/Dictionary_of_Architecture_and_Building_Construction
4. Favorite (Clever) Defensive Programming Best Practices [closed] https://stackoverflow.com/questions/490420/favorite-clever-defensive-programming-best-practices
5. Why AI Writes Good Web Code but fails with Embedded C++? | Fadi ... https://www.linkedin.com/posts/fadi-labib_why-ai-writes-good-web-code-but-fails-with-activity-7425448381050757120-CAD-
6. Alejandro Benítez-Llambay - LinkedIn https://www.linkedin.com/posts/alejandro-benitez-llambay_camels-illustristng-simulation-activity-7446504427743637504-1otj
7. ConGen—A Simulator-Agnostic Visual Language for Definition and ... https://pmc.ncbi.nlm.nih.gov/articles/PMC8777257/

8. [PDF] Objaverse: A Universe of Annotated 3D Objects - CVF Open Access https://openaccess.thecvf.com/content/CVPR2023/papers/Deitke_Objaverse_A_Universe_of_Annotated_3D_Objects_CVPR_2023_paper.pdf
9. bing.txt - FTP Directory Listing ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/bing.txt
10. Steam 创意工坊::Jarlies NEWER mods https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=3032266079
11. [01/53] [abbrv] english analyzer namespace finished-Apache Mail ... https://lists.apache.org/thread/4s4my6jn4rd9kxm3fhtz0k75w14prb2y
12. Champions of Norrath Preview | PDF | Indemnity | Video Games https://www.scribd.com/document/899354510/Game-Informer-Issue-123-July-2003
13. (PDF) Cloud Atlas: A Novel - Academia.edu https://www.academia.edu/18494008/Cloud_Atlas_A_Novel
14. Dolmenwood - Players Book (Necrotic Gnome) (Z-Library) - Scribd https://www.scribd.com/document/768927141/Dolmenwood-Players-Book-Necrotic-Gnome-Z-Library
15. Steam 创意工坊::ZeR0-N's Sub items https://steamcommunity.com/sharedfiles/filedetails/?l=schinese&id=2598238129
16. Word Play from Think Python 2e - Kaggle https://www.kaggle.com/code/priteshshrivastava/word-play-from-think-python-2e
17. 333333 23135851162 the 13151942776 of 12997637966 ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt
18. eng_wikipedia_2012_3M-sentences.txt - Apache https://svn.apache.org/repos/bigdata/opennlp/trunk/leipzig/data/eng_wikipedia_2012_3M-sentences.txt
19. C++: Designing a component-based entity system - Stack Overflow https://stackoverflow.com/questions/5119344/c-designing-a-component-based-entity-system-advanced-problems
20. largedictionary.txt - CS@Columbia http://www.cs.columbia.edu/~allen/S14/HMWK/largedictionary.txt
21. 虚幻引擎5.7版本说明 https://dev.epicgames.com/documentation/unreal-engine/unreal-engine-5-7-release-notes?lang=zh-CN
22. There are many misconceptions about CQRS. | Milan Jovanović https://www.linkedin.com/posts/milan-jovanovic_there-are-many-misconceptions-about-cqrs-activity-7378392542465265664-aAV9
23. [PDF] emnlp-ijcnlp 2019 - ACL Anthology https://aclanthology.org/D19-3.pdf
24. Past Groups - W3C https://www.w3.org/community/groups/past/

25. [PDF] Middleware and programming models for multi-robot systems - HAL https://hal.science/tel-01809505/file/these.pdf
26. Creating a cluster > Setting up replication > Cluster recovery https://manual.manticoresearch.com/Creating_a_cluster/Setting_up_replication/Cluster_recovery
27. C++ Object Persistence with ODB 转载 - CSDN博客 https://blog.csdn.net/adrian169/article/details/8996575
28. [PDF] z/OS V2R2 Common Information Model User's Guide - IBM https://publibz.boulder.ibm.com/epubs/pdf/cfz2ug10.pdf
29. 1 - Amazon S3 https://s3.eu-central-1.amazonaws.com/deepset.ai-farm-models/0.4.1/glove-english-uncased-6B/vocab.txt
30. Event-based Game engine based on polymorphism of Entities https://stackoverflow.com/questions/29583579/event-based-game-engine-based-on-polymorphism-of-entities
31. Game++. Part 1.1: C++, game engines, and architectures ... https://dev.to/pvsdev/game-part-11-c-game-engines-and-architectures-bif
32. Unity 2023.2.0b16 https://unity.com/cn/releases/editor/beta/2023.2.0b16
33. Multi-agent framework for schema-guided reasoning and tool ... https://www.sciencedirect.com/science/article/pii/S0926580526001299
34. Add a shader pass in a custom shader - Unity - Manual https://docs.unity3d.com/6000.3/Documentation/Manual/writing-shader-create-shader-pass.html
35. #angular | Roberto Heckers | 14 comments - LinkedIn https://www.linkedin.com/posts/roberto-heckers-2313453b_angular-activity-7422919977193881600-VtMd